

PocketVault

临时文件交接箱

学习内容去重 用户隔离和限次分享的并发语义

从零运行 · 源码讲解 · 故障实验 · 30 个面试追问

配套源码 <https://github.com/xr-xiaoran/pocketvault>

2026年9月14日 | 个人学习项目 | AI 辅助实现

目录

目录	2
1 项目场景和两处小设计	3
2 从零运行	3
3 手动完成一个业务闭环	3
4 从请求追到数据库	4
5 Go 基础在这里怎样用	4
6 三张表的职责	5
7 去重怎样避免并发重复插入	5
8 空间账本怎样计算	5
9 分享凭证为什么只存哈希	6
10 并发限次下载的正确做法	6
11 HEAD Range 撤销和删除的细节	6
12 安全响应头和身份边界	6
13 你必须亲手完成的实验	7
14 七天学习安排	7
15 面试问题与参考回答	8
16 两分钟介绍与验收标准	11
验证记录	11
关键源码逐行对照	11

哥哥，这份手册对应 github.com/xr-xiaoran/pocketvault。建议用你自己的临时文本文件练习，依次完成上传、去重、分享、耗尽、撤销和并发实验。重点是弄懂内容寻址、所有权、事务和有限次数凭证，不是把它称为已经服务大量用户的网盘。

1 项目场景和两处小设计

协会或小队经常临时交接文件：同一份材料被多次上传，文件名不同但内容相同；分享链接发出去后，希望只允许有限次取走，并能主动撤销。PocketVault 为这个小场景实现文件存储和短时分享。

第一处小设计是“空间账本”：同一用户上传相同字节两次，生成两个文件记录，但只保存一份内容。统计接口同时显示逻辑字节数、物理内容字节数和节省字节数，面试时能直接演示去重效果。

第二处是“可撤销下载凭证”：创建分享时指定次数和过期秒数；每次被服务接纳的 GET 消耗一次。次数耗尽、已过期、已撤销都不再放行。不要理解成用户下载后文件会从世界上消失，也不能阻止已获得文件的人复制。

当前范围是每文件最大 4 MiB、每用户物理内容最多 64 MiB、最多 1000 个逻辑文件的单机学习项目。这里的小文件限制是实现边界，不是性能成绩。

2 从零运行

终端 A 进入项目目录：

```
go version
go mod download
go test -count=1 -cover ./...
go vet ./...
$env:VAULT_KEYS='alice=alice-local-demo-key-12345,bob=bob-local-demo-key-123456'
go run ./cmd/pocketvault
```

终端 B：

```
$env:VAULT_KEY='alice-local-demo-key-12345'
powershell -ExecutionPolicy Bypass -File ./scripts/demo.ps1
```

VAULT_KEYS 是服务端信任的身份配置，VAULT_KEY 是演示脚本使用的一个 key。它们名字不同，因为前者包含多个用户映射，后者只代表当前演示者。这里没有注册登录，没有 JWT；不应在简历上给本项目添加不存在的登录功能。

默认监听 127.0.0.1:8082；DB_PATH 不设置时自动创建 pocketvault.db。默认 key 只是示例。你可以生成自己的长随机值，但两个终端的配置必须相符。

demo 做两次相同内容上传、创建一次性分享、下载一次、验证第二次 410、展示空间统计。每次运行使用新的随机内容，避免受上一轮数据干扰。文件会保留，最多 1000 个文件后需通过 API 删除不需要的文件。

3 手动完成一个业务闭环

先建立请求头和上传字节：

```
$h=@{Authorization="Bearer $env:VAULT_KEY"}
$bytes=[System.Text.Encoding]::UTF8.GetBytes('hello pocketvault')
$a=Invoke-RestMethod -Method Post -Uri 'http://127.0.0.1:8082/api/files?name=notes.txt' -Headers $h -ContentType 'application/octet-stream' -Body $bytes
$a | ConvertTo-Json -Depth 6
```

响应中 file.id 是逻辑文件 ID, file.sha256 是内容摘要, size 是字节数, deduplicated 表示本次是否复用已有内容。再次用不同文件名上传相同 bytes, 应得到不同 file.id、相同 sha256、deduplicated=true。

创建分享:

```
$body=@{downloads=1;ttl_seconds=60}|ConvertTo-Json
$s=Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:8082/api/files/$(a.file.id)/shares" -Headers $h -ContentType 'application/json' -Body $body
$url="http://127.0.0.1:8082/s/$(s.token)"
Invoke-WebRequest -UseBasicParsing -Uri $url
```

再次请求同一 URL 应返回 410。这里访问 share URL 不需要原用户 API key, 因为 token 本身就是授权凭证。把 URL 交给谁, 谁就可能使用它。浏览器插件或安全扫描器如果真的发 GET 也会花掉次数, 不能把它误说成“精确允许一个人下载”。

统计和撤销:

```
Invoke-RestMethod -Uri 'http://127.0.0.1:8082/api/stats' -Headers $h
Invoke-RestMethod -Uri 'http://127.0.0.1:8082/api/shares' -Headers $h
Invoke-RestMethod -Method Delete -Uri "http://127.0.0.1:8082/api/shares/$(s.id)" -Headers $h
```

列表不返回明文 token。创建分享时拿到 token 后应妥善保存; 遗失时新建分享即可, 服务端不能从摘要还原它。

4 从请求追到数据库

上传请求 -> API key 映射 owner -> 限制读取 4 MiB
-> 校验名字 -> SHA-256 内容摘要
-> 事务插入或复用 blob -> 检查配额 -> 新建 file -> 提交

创建分享 -> owner 条件校验 file -> 生成随机 token
-> 只保存 token 摘要与次数/过期时间

下载 -> 摘要查询 token -> 条件更新 remaining-1
-> 在同一事务读取字节 -> 提交 -> attachment 响应

cmd/pocketvault/main.go 解析 VAULT_KEYS, 拒绝短 key、重复 key 和重复 owner, 避免身份歧义。
internal/httpapi/api.go 处理身份、上传正文和 HTTP 头。internal/vault/store.go 负责原子数据操作。

目录虽小, 但业务责任明确。Store 接收 owner 参数; owner 的可信来源在 API 层。直接调用 Store 的测试可以指定 alice/bob, 这是测试准备, 不表示真实客户端能伪造 owner。

5 Go 基础在这里怎样用

`[]byte` 表示原始字节序列, 适合文件内容。string 也能保存字节, 但文件不必是合法 UTF-8, 因此正文按 bytes 处理。文件名则要求合法 UTF-8, 且不能包含斜杠、反斜杠、换行或 NUL。

`io.ReadAll` 会把数据读入内存。这里只允许最大 4 MiB, 因此采取简单的有界读入; 它不是大文件流式上传。多个并发上传会消耗“并发数乘文件大小”的内存, 大规模服务还要限制并发并改为流式方案。

`http.MaxBytesReader` 在 HTTP 层限制大小, Store.Put 再检查 `len(data)`, 形成不同调用入口都能遵守的规则。不能只相信 Content-Length, 因为客户端可能分块传输或发送错误长度。

`crypto/rand` 用于生成难以预测的 token; 普通数学随机数适合模拟和退避, 不适合授权凭证。`crypto/sha256` 用于内容摘要和 token 摘要, 两者调用相同算法, 但目的不同。

sql.Tx 是一组要一起成功或失败的 SQL 操作。事务没有跨文件系统边界，因此把 BLOB 放 SQLite 内部，便于初学者清楚验证原子性。将来迁移对象存储时，这个简化就不再成立。

6 三张表的职责

blobs 表的主键为 (owner, hash)，保存 data 和 size。相同用户、相同内容只能一份；不同用户保存相同内容也各自占一份，避免跨用户泄露“别人是否上传过该内容”。

files 表保存逻辑记录：id、owner、name、hash、size、created_at。两个名字可以指向相同 blob。文件 ID 与哈希不是一回事；修改文件名不应该改变内容地址，而上传修改过的内容会得到新哈希。

shares 表引用 file_id，保存 token_hash、remaining、expires_at、revoked。file_id 外键使用 ON DELETE CASCADE；删除逻辑文件时自动清除相应分享。删除某个逻辑文件不会影响另一个指向相同 blob 的逻辑文件。

时间使用 Unix 秒；TTL 取值 1-86400 秒，下载次数 1-100。若 expires_at 等于当前秒数，已经过期，必须拒绝。和 RetryLab 的毫秒字段不同，不能把两个项目的时间值直接混用。

7 去重怎样避免并发重复插入

上传首先计算 SHA-256，然后事务内执行：

```
INSERT INTO blobs(owner,hash,data,size)
VALUES(?,?,?,?)
ON CONFLICT(owner,hash) DO NOTHING;
```

数据库唯一约束解决两个请求同时上传相同内容的竞争。影响行数为 0，说明复用了已有 blob；为 1 则新建。随后在同一事务检查用户物理内容大小和文件数量，再插入 files。

为什么先写再查配额：SQLite 的 deferred 事务如果先读后写，多个连接可能同时持有读快照再尝试升级 writer，产生忙/升级失败。先执行写语句取得写事务，再查询配额和写元数据。超限时整体回滚，刚插入的 blob 也会撤销。

这里只做上传后的存储去重，不是“客户端只上传哈希就秒传”。上传完整字节才能由服务端验证内容地址，也避免把哈希存在性当成访问权限。SHA-256 理论上仍可能碰撞；当前采用常见的密码学碰撞可忽略假设，没有实现逐字节碰撞分桶。

8 空间账本怎样计算

例如内容长度为 100 字节，alice 上传两次：files=2、blobs=1、logical_bytes=200、physical_bytes=100、saved_bytes=100。再传一份 50 字节不同内容，逻辑总计 250、物理内容总计 150。

physical_bytes 表示现存 blob 的内容长度总和，不是 Windows 资源管理器显示的 .db 文件大小。数据库索引、页面、WAL 日志、空闲页都会占磁盘。删除 blob 后统计下降，.db 不一定马上缩小，这不代表删除失败。

Stats 使用一条 SQL 获取四项子查询结果，减少分开查询期间插入/删除导致不同时间点统计混合的问题。saved_bytes 等于 logical_bytes 减 physical_bytes。

在面试中展示这个例子比写“空间节省 50%”更准确。这个 50% 只适用于两份完全相同内容的示例，不是所有用户数据都能节省一半。

9 分享凭证为什么只存哈希

token 由 32 个安全随机字节组成，十六进制编码后长 64 个字符，随机性仍是 256 位，字符数增加不等于熵增加。返回原文 token 给创建者，数据库只保存 SHA-256(token)。

这样数据库表泄露时，攻击者不能直接把 token_hash 复制到下载 URL 使用，因为服务会再对输入做一次哈希。安全基础是 token 本身具有高熵；弱密码不能用“直接 SHA-256”代替 bcrypt/Argon2。

明文 token 仍会出现在分享 URL，可能进入浏览历史、代理日志和聊天记录。本项目不写访问 URL 日志，响应设置 no-referrer、no-store；真实部署还要 HTTPS、日志脱敏和权限管理。数据库内容没有加密，不能把哈希存 token 说成文件已加密。

10 并发限次下载的正确做法

错误思路：先 SELECT remaining, if remaining>0 就返回文件，然后 UPDATE remaining-1。两个请求都可能看见 remaining=1，最终放行两次。

实际代码先执行原子条件扣减：

```
UPDATE shares SET remaining=remaining-1
WHERE token_hash=? AND revoked=0
  AND expires_at>? AND remaining>0
RETURNING file_id;
```

没有返回行，统一表示不存在、过期、撤销或耗尽，API 返回 410。返回 file_id 后在同一事务读取 bytes，再 Commit；数据库查询失败时回滚，次数不会丢失。

HTTP 写入发生在 Commit 之后。如果网络断开，数据库已经扣了一次。项目明确定义额度是“获准 GET 的次数”，不是“完整下载成功的次数”。没有可靠客户端确认协议时，服务端无法知道用户是否完整收到并保存了文件。自动补回次数还可能让恶意断连者无限获得数据。

因此成功条件是数据库放行次数不超额度，测试的 40 争 3 正是在验证这一点。它不能证明互联网传输 exactly-once，也不证明只会有 3 个不同的人看过文件。

11 HEAD Range 撤销和删除的细节

Go 的 GET 路由也可能匹配 HEAD。本项目在下载 handler 里检查 Method，只允许 GET，否则 405，确保探测请求不会悄悄扣次数。

Range 断点请求返回 416，不消耗次数。因为一次完整文件下载可能变成多个分段请求，必须先定义“次数按请求还是按会话”。当前选择简单规则，不支持断点续传。

撤销将 revoked 设为 1，而且要校验 share 所属 file 的 owner。重复撤销仍然安全。撤销先于后续下载事务完成时，后续会被拒绝；已在撤销之前成功获得授权并读取了 bytes 的请求仍可能完成发送。

删除 files 采用 owner 条件，拿到 hash 后检查是否还有其他引用；没有时删除 blob。files、shares、blob 的变化在同一数据库事务内完成。不同于磁盘文件加数据库的双存储方案，不需要先删文件再删行。

12 安全响应头和身份边界

Content-Type 使用 application/octet-stream，Content-Disposition 使用 attachment，并通过 mime.FormatMediaType 编码文件名；浏览器按附件处理，减少上传 HTML/SVG 后被站内页面执行的风险。

X-Content-Type-Options: nosniff 告诉浏览器不要猜类型；Cache-Control: no-store, private 降低中间缓存保存短期私密文件的机会；Referrer-Policy: no-referrer 避免分享凭证随来源头传播。

这些响应头不是访问控制的替代品。真正的后台权限来自 Bearer key 对 owner 的映射，下载权限来自 share token。用户 B 删除或分享用户 A 的文件返回 404，既不允许操作，也不向 B 明确泄露文件存在性。

当前每个 key 是操作员预先配置的，尚无注册登录、密码恢复、动态 key 吊销管理和审计平台。涉及这些模块时要追加设计与测试，不能在面试中补写不存在的代码。

13 你必须亲手完成的实验

实验	操作	正确结果
基础上传	发送小文本字节	201, 大小与字节数一致
内容去重	同用户相同内容不同名字	不同文件 ID, 同哈希, 一个 blob
隔离去重	换 bob 上传相同内容	不返回跨用户复用状态
限一次	downloads=1 连续 GET 两次	200 后 410
到期边界	now 恰好等于 expires_at	ErrGone
撤销	未用完即 DELETE share	后续下载 410
HEAD	在第一次 GET 前 HEAD	405, 不扣次数
越权	bob 分享 alice 文件	404
删除引用	两个文件同 blob, 删一个	另一个仍可下载
并发抢额度	40 请求、3 次额度、两个连接	恰好 3 次成功
重启	重开同数据库再下载	字节仍相同
大小限制	超过 4 MiB	413, 不保存大文件

查看 internal/vault/store_test.go 和 internal/httpapi/api_test.go。前者含真实数据库事务，后者走真实本地 HTTP 服务。测试不是外部公网部署，也没有压测线上用户行为。

14 七天学习安排

第 1 天：运行 demo，手动上传和创建分享，记录每个请求的状态码。搞清 bytes、UTF-8、JSON、请求头和 URL query 分别存什么。

第 2 天：手画 blobs/files/shares 三张表，解释为什么两个文件名可以共用一份内容。写一段 Go 程序计算 hello 的 SHA-256，修改一个字符观察输出。

第 3 天：重写 Put 和 Stats；补相同内容两次上传测试，删除一个引用后检查统计。解释为何物理内容字节不等于数据库文件大小。

第 4 天：重写 Share；阅读 crypto/rand、sha256、hex，明确 token 只返回一次。给分享列表加一个测试：不出现 raw token。

第 5 天：重写 Consume；先故意做先查再减版本，讨论它可能出现什么竞态，再改为条件更新并通过并发测试。不要把错误实现发布成正确版本。

第 6 天：理解 Delete/Revoke 和外键；补 HEAD、Range 不扣次数的 HTTP 测试，测试数据库出错如何返回。最后运行 go vet 和全部测试。

第 7 天：选一个真实改进，例如按文件名过滤列表或添加分享备注，补权限和边界测试；提交自己的 commit，录下 3 分钟演示并准备下面的追问。

15 面试问题与参考回答

1 你为什么做这个项目

我希望解决小团队临时文件交接中的重复存储和分享失控问题。项目用去重空间账本展示存储复用，用限次且可撤销的凭证限制后续访问。当前是单机小文件学习实现。

2 为什么不用电商或通用网盘

这个场景让我集中学习文件字节、内容哈希、事务、授权凭证和并发扣额度，功能少但能把边界讲清。不要说市场没有类似产品；辨识度来自可演示的组合和实现说明。

3 为什么选 SHA-256

希望内容地址有较低碰撞风险且容易跨语言验证。普通非密码学哈希适合散列表，MD5 不适合作为抗恶意碰撞的安全依据。当前不把 hash 当用户权限。

4 哈希相同就绝对相同吗

有限长度哈希理论上一定有碰撞。工程中 SHA-256 碰撞可忽略假设适用于这里；如果业务要求进一步防御，可以同长度再逐字节比较、增加碰撞处理分支。当前代码未实现这种分支。

5 文件 ID 和 blob hash 为什么分开

文件 ID 代表一个用户的逻辑记录及名字、分享关系；hash 代表内容。两个逻辑文件可以指向同一内容，删除一个不应删掉另一个的内容。

6 为什么只在同用户内去重

全局去重会让 deduplicated 响应或统计成为存在性探针，暴露别人是否拥有某份内容。当前使用 owner+hash，牺牲跨用户节省，简化隔离边界。

7 这是秒传吗

不是。客户端发送完整 bytes，服务端校验哈希并复用存储。秒传通常需要上传前查询和持有证明协议，不能仅凭哈希给用户增加文件访问权。

8 为什么把文件放 SQLite

小文件场景中，BLOB 和元数据能够一个事务提交，避免文件系统与数据库分别提交的崩溃窗口。代价是数据库变大、写串行和读入内存；不适合直接扩展成大规模网盘。

9 如果改用 S3/MinIO 怎么办

先上传临时对象，元数据记录 pending，完成后确认 ready；失败任务异步回收孤儿对象，删除可走 outbox/后台重试。需要设计恢复流程，因为数据库事务不能回滚一个已完成的对象存储请求。

10 为什么不能相信 Content-Length

请求可以分块发送，字段也可能错误。服务端必须实际限制读取字节数。这里 MaxBytesReader 限制 HTTP，Store 再检查长度，正常大小才进入持久化。

11 这是不是流式上传

不是。这里有上限地读入最多 4 MiB，简单且容易测试。若处理大文件，应流式复制、边读边 hash，并限制总量、速率和并发，避免把整份文件装入内存。

12 分享凭证有多少随机性

32 个随机字节等于 256 位。十六进制后是 64 字符，只是表示方法改变，熵没有翻倍。random 来自 crypto/rand，而不是 math/rand。

13 为什么只存 token 的 SHA-256

数据库泄露时不直接暴露可使用的下载凭证。请求到来时计算摘要后匹配。由于 token 高熵，直接摘要适合这里；用户自选弱密码应使用专门密码哈希算法。

14 URL 泄露了还能保护吗

持有 URL 的人可能消费剩余次数，因此 token 泄露就是授权泄露。可立即撤销，过期和额度也限制风险，但已经下载的副本无法追回。TLS 和日志脱敏仍有必要。

15 为什么不用 JWT 做下载链接

限次和主动撤销都需要服务端状态。这里直接用随机凭证查数据库即可，JWT 不会自动解决次数扣减和撤销，反而会引入签名密钥管理。

16 如何保证限次不超卖

把 remaining>0、未过期、未撤销都写在同一条 UPDATE WHERE 中，并用 RETURNING 判断是否放行。不能先读取 remaining 再在 Go 中判断后减一。

17 40 个请求争 3 次意味着什么

本地两个独立数据库句柄、40 个并发调用只有 3 个成功，其他收到 ErrGone。这证明该测试场景下原子额度正确，不代表达到某个 QPS，也不代表经过公网压力测试。

18 下载中途断开会退回次数吗

不会。数据库提交后额度就花掉，语义是授权次数。网络完成与否没有可靠的跨端原子提交。若要重新取，可由拥有者另外创建凭证，而不是自动补回造成重用漏洞。

19 HEAD 为什么特殊处理

HEAD 只是请求响应头，不应读取业务文件并扣额度。Go GET 路由可能匹配 HEAD，所以 handler 显式拒绝，测试先 HEAD 再 GET 证明没有消耗。

20 Range 为什么暂不支持

分段下载会把一次用户下载拆成多个 HTTP 请求。必须先设计会话和次数语义。当前直接 416，并且在 Consume 前判断，避免无效请求扣额度。

21 撤销与正在下载同时发生怎么办

数据库写事务决定谁先获得授权。撤销后新授权被拒绝，已完成扣减并读取 bytes 的请求可能仍然发出内容。撤销不是撤回已发送字节。

22 删除文件为什么不会误删共享内容

先按 owner 删除逻辑文件，再检查同 owner/hash 是否还有 files 引用；有就保留 blob，没有才删除。所有操作同事务，外键删除 shares。

23 配额怎么防并发突破

事务第一条是写操作，再查询总大小和文件数，超限则回滚。其他 writer 不会在配额检查和提交之间插入自己的成功写入。当前配额限制文件内容与条目，分享历史数量的清理策略尚未实现。

24 为什么统计下降了数据库还很大

SQLite 删除数据会留下可复用空闲页，WAL 和索引也占空间。统计只反映存活内容；需要离线维护或 VACUUM 才可能回收文件体积，不能在每次删除时都做重维护。

25 文件名怎样避免目录穿越

这里不按用户名字写磁盘路径，名字只作元数据，还拒绝斜杠、反斜杠、NUL 和换行。迁移文件系统后仍应通过服务端生成的安全路径写入，不能拼接任意客户端路径。

26 Content-Disposition 有什么作用

要求浏览器以附件下载，使用标准库安全格式化文件名；配合 octet-stream 和 nosniff，减少恶意 HTML 当同源页面执行。没有病毒扫描，不能宣称文件内容本身安全。

27 用户身份从哪里来

操作员预置 key 映射 owner。客户端提供 Bearer key，服务端通过恒定时间比较校验。不是从 user_id query 或 X-User 直接读身份。项目没实现密码登录。

28 哪些数据可能是秘密

API key、原始 share token 和文件内容都要保护。数据库只保存 token 摘要，但 BLOB 未加密；备份也含敏感内容。源码仓库不应提交数据库或真实 key。

29 下一步你最想改什么

先增加限流和并发内存上限、分享数量配额与过期记录清理，再按真实需求考虑对象存储。扩展必须加测试，尤其不能为了大文件而无上限地 ReadAll。

30 简历如何描述才准确

写“实现用户内 SHA-256 去重与逻辑文件引用分离、可撤销限次分享、条件更新防止额度并发超扣，并用真实 SQLite 和 HTTP 测试验证”。不写分布式存储、分块秒传、端到端加密或百万用户。

16 两分钟介绍与验收标准

介绍：我做了一个 Go 临时文件交接服务，重点是相同内容复用和分享权限收回。文件逻辑记录和内容分表，按 owner 加 SHA-256 去重，用事务保证字节、元数据、配额一致。分享 token 使用安全随机数，数据库只存摘要；下载时原子扣次数，同时判断过期和撤销。项目有空间统计与一键演示，测试包括两连接 40 请求争 3 次额度。当前是小文件单机实现，额度按授权请求计算，没有分块上传和对象存储。

现场演示先上传同样内容两次，解释统计；创建一次性分享，展示 200 后 410；创建新分享后撤销，展示拒绝；最后打开并发测试，解释错误实现为什么会多放行。

完成标准：自己实现 Put、Share、Consume；不看稿解释 transaction、hash、owner、token；通过全部测试与 demo；亲手补一个 Range 不扣额度测试或文件过滤功能；能说明至少三项限制。掌握后再在投递简历中展开项目，避免把“读过代码”当成“能维护代码”。

参考入口：<https://pkg.go.dev/crypto/rand>，<https://pkg.go.dev/crypto/sha256>，<https://pkg.go.dev/net/http>，https://www.sqlite.org/lang_returning.html，<https://www.sqlite.org/wal.html>。文档里的并发语义以当前实现和测试为准。

验证记录

本机 Windows Go 1.26.5 完成真实 SQLite 测试、HTTP 测试、go vet、go build 与演示脚本；GitHub Actions Linux 已通过包含竞态检测的工作流。后续提交状态请查看仓库 Actions 页面。

详细测试证据见仓库 TESTING.md；覆盖率不等于所有情况都正确，并发测试数据不作为吞吐量成绩。

关键源码逐行对照

以下为当前实现的关键文件，行号对应生成时的源码。PDF 中长行会折行，复制运行请使用仓库原文件。阅读时先解释每个函数的输入、输出、错误及数据库副作用。

internal/vault/store.go

```

001 // Package vault implements tenant-scoped content-addressed storage and revocable tickets.
002 package vault
003
004 import (
005     "context"
006     "crypto/rand"
007     "crypto/sha256"
008     "database/sql"
009     "encoding/hex"
010     "errors"
011     "fmt"
012     _ "modernc.org/sqlite"
013     "strings"
014     "time"
015     "unicode/utf8"
016 )
017
018 const MaxBytes = 4 << 20
019
020 var (
021     ErrInvalid = errors.New("invalid input")
022     ErrNotFound = errors.New("not found")
023     ErrGone     = errors.New("share expired, revoked, or exhausted")
024     ErrQuota    = errors.New("owner quota exceeded")
025 )
026
027 type Store struct{ db *sql.DB }
028 type File struct {
029     ID      int64 `json:"id"`
030     Name    string `json:"name"`
031     Hash    string `json:"sha256"`
032     Size    int64 `json:"size"`
033     Created int64 `json:"created_at"`
034 }
035 type Share struct {
036     ID      int64 `json:"id"`
037     Token   string `json:"token,omitempty"`
038     FileID  int64 `json:"file_id"`
039     Remaining int  `json:"remaining"`
040     Expires int64 `json:"expires_at"`
041     Revoked  bool  `json:"revoked"`
042 }
043 type Stats struct {
044     Files      int64 `json:"files"`
045     Blobs      int64 `json:"blobs"`
046
047     LogicalBytes int64 `json:"logical_bytes"`
048     PhysicalBytes int64 `json:"physical_bytes"`
049     SavedBytes    int64 `json:"saved_bytes"`
050 }
051 func Open(path string) (*Store, error) {
052     db, err := sql.Open("sqlite", path)
053     if err != nil {
054         return nil, err
055     }
056     db.SetMaxOpenConns(1)
057     _, err = db.Exec(`PRAGMA busy_timeout=5000; PRAGMA journal_mode=WAL; PRAGMA foreign_keys=ON;
058 CREATE TABLE IF NOT EXISTS blobs(owner TEXT NOT NULL,hash TEXT NOT NULL,data BLOB NOT NULL,size
INTEGER NOT NULL CHECK(size>0),PRIMARY KEY(owner,hash));
059 CREATE TABLE IF NOT EXISTS files(id INTEGER PRIMARY KEY,owner TEXT NOT NULL,name TEXT NOT
NULL,hash TEXT NOT NULL,size INTEGER NOT NULL,created_at INTEGER NOT NULL,
060 FOREIGN KEY(owner,hash) REFERENCES blobs(owner,hash));`);

```

```

061 CREATE INDEX IF NOT EXISTS files_owner_hash ON files(owner,hash);
062 CREATE TABLE IF NOT EXISTS shares(id INTEGER PRIMARY KEY,file_id INTEGER NOT NULL REFERENCES
files(id) ON DELETE CASCADE,
063 token_hash TEXT NOT NULL UNIQUE,remaining INTEGER NOT NULL CHECK(remaining>=0),expires_at
INTEGER NOT NULL,revoked INTEGER NOT NULL DEFAULT 0 CHECK(revoked IN (0,1)));
064 CREATE INDEX IF NOT EXISTS shares_file ON shares(file_id);`)
065     if err != nil {
066         db.Close()
067         return nil, err
068     }
069     return &Store{db: db}, nil
070 }
071 func (s *Store) Close() error { return s.db.Close() }
072 func (s *Store) Ping(ctx context.Context) error { return s.db.PingContext(ctx) }
073 func digest(data []byte) string { v := sha256.Sum256(data); return
hex.EncodeToString(v[:]) }
074 func validName(name string) bool {
075     return len(name) > 0 && len(name) <= 180 && utf8.ValidString(name) &&
!strings.ContainsAny(name, "/\\r\\n\\x00") && strings.TrimSpace(name) != ""
076 }
077
078 // Put keeps bytes and metadata in the same database transaction: no orphan file window.
079 func (s *Store) Put(ctx context.Context, owner, name string, data []byte, now time.Time) (File,
bool, error) {
080     if owner == "" || !validName(name) || len(data) == 0 || len(data) > MaxBytes {
081         return File{}, false, ErrInvalid
082     }
083     hash := digest(data)
084     tx, err := s.db.BeginTx(ctx, nil)
085     if err != nil {
086         return File{}, false, err
087     }
088     defer tx.Rollback()
089     // First statement is a write, avoiding deferred read-to-write upgrade races.
090     r, err := tx.ExecContext(ctx, `INSERT INTO blobs(owner,hash,data,size) VALUES(?,?,?,?) ON
CONFLICT(owner,hash) DO NOTHING`, owner, hash, data, len(data))
091     if err != nil {
092         return File{}, false, err
093     }
094     n, err := r.RowsAffected()
095     if err != nil {
096         return File{}, false, err
097     }
098     var total, count int64
099     if err = tx.QueryRowContext(ctx, `SELECT COALESCE(SUM(size),0) FROM blobs WHERE owner=?`,
owner).Scan(&total); err != nil {
100         return File{}, false, err
101     }
102     if err = tx.QueryRowContext(ctx, `SELECT COUNT(*) FROM files WHERE owner=?`,
owner).Scan(&count); err != nil {
103         return File{}, false, err
104     }
105     if total > 64<<20 || count >= 1000 {
106         return File{}, false, ErrQuota
107     }
108     r, err = tx.ExecContext(ctx, `INSERT INTO files(owner,name,hash,size,created_at)
VALUES(?,?,?,?,?)`, owner, name, hash, len(data), now.Unix())
109     if err != nil {
110         return File{}, false, err
111     }
112     id, err := r.LastInsertId()
113     if err != nil {
114         return File{}, false, err
115     }

```

```

116     f := File{id, name, hash, int64(len(data)), now.Unix()}
117     return f, n == 0, tx.Commit()
118 }
119 func (s *Store) List(ctx context.Context, owner string) ([]File, error) {
120     rows, err := s.db.QueryContext(ctx, `SELECT id,name,hash,size,created_at FROM files WHERE
    owner=? ORDER BY id DESC LIMIT 1000`, owner)
121     if err != nil {
122         return nil, err
123     }
124     defer rows.Close()
125     out := make([]File, 0)
126     for rows.Next() {
127         var f File
128         if err = rows.Scan(&f.ID, &f.Name, &f.Hash, &f.Size, &f.Created); err != nil {
129             return nil, err
130         }
131         out = append(out, f)
132     }
133     return out, rows.Err()
134 }
135
136 func (s *Store) Stats(ctx context.Context, owner string) (Stats, error) {
137     var st Stats
138     err := s.db.QueryRowContext(ctx, `SELECT (SELECT COUNT(*) FROM files WHERE owner=?),
139 (SELECT COUNT(*) FROM blobs WHERE owner=?),(SELECT COALESCE(SUM(size),0) FROM files WHERE
    owner=?),
140 (SELECT COALESCE(SUM(size),0) FROM blobs WHERE owner=?`, owner, owner, owner,
    owner).Scan(&st.Files, &st.Blobs, &st.LogicalBytes, &st.PhysicalBytes)
141     st.SavedBytes = st.LogicalBytes - st.PhysicalBytes
142     return st, err
143 }
144 func (s *Store) Delete(ctx context.Context, owner string, id int64) error {
145     tx, err := s.db.BeginTx(ctx, nil)
146     if err != nil {
147         return err
148     }
149     defer tx.Rollback()
150     var hash string
151     err = tx.QueryRowContext(ctx, `DELETE FROM files WHERE id=? AND owner=? RETURNING hash`, id,
    owner).Scan(&hash)
152     if errors.Is(err, sql.ErrNoRows) {
153         return ErrNotFound
154     }
155     if err != nil {
156         return err
157     }
158     _, err = tx.ExecContext(ctx, `DELETE FROM blobs WHERE owner=? AND hash=? AND NOT
    EXISTS(SELECT 1 FROM files WHERE owner=? AND hash=?`, owner, hash, owner, hash)
159     if err != nil {
160         return err
161     }
162     return tx.Commit()
163 }
164 func (s *Store) Share(ctx context.Context, owner string, fileId int64, downloads, ttl int, now
    time.Time) (Share, error) {
165     if downloads < 1 || downloads > 100 || ttl < 1 || ttl > 86400 {
166         return Share{}, ErrInvalid
167     }
168     tokenBytes := make([]byte, 32)
169     if _, err := rand.Read(tokenBytes); err != nil {
170         return Share{}, err
171     }
172     token := hex.EncodeToString(tokenBytes)

```

```

173     tx, err := s.db.BeginTx(ctx, nil)
174     if err != nil {
175         return Share{}, err
176     }
177     defer tx.Rollback()
178     // INSERT SELECT binds authorization and creation to one statement.
179     r, err := tx.ExecContext(ctx, `INSERT INTO shares(file_id,token_hash,remaining,expires_at)
180 SELECT id,?,?,? FROM files WHERE id=? AND owner=?`, digest([]byte(token)), downloads,
    now.Unix()+int64(ttl), fileID, owner)
181     if err != nil {
182         return Share{}, err
183     }
184     n, err := r.RowsAffected()
185     if err != nil {
186         return Share{}, err
187     }
188     if n != 1 {
189         return Share{}, ErrNotFound
190     }
191     id, err := r.LastInsertId()
192     if err != nil {
193         return Share{}, err
194     }
195     return Share{id, token, fileID, downloads, now.Unix() + int64(ttl), false}, tx.Commit()
196 }
197 func (s *Store) Revoke(ctx context.Context, owner string, id int64) error {
198     r, err := s.db.ExecContext(ctx, `UPDATE shares SET revoked=1 WHERE id=? AND file_id
199 IN(SELECT id FROM files WHERE owner=?)`, id, owner)
200     if err != nil {
201         return err
202     }
203     n, err := r.RowsAffected()
204     if err != nil {
205         return err
206     }
207     if n != 1 {
208         return ErrNotFound
209     }
210     return nil
211 }
212 func (s *Store) Shares(ctx context.Context, owner string) ([]Share, error) {
213     rows, err := s.db.QueryContext(ctx, `SELECT
214 s.id,s.file_id,s.remaining,s.expires_at,s.revoked FROM shares s JOIN files f ON f.id=s.file_id
215 WHERE f.owner=? ORDER BY s.id DESC LIMIT 100`, owner)
216     if err != nil {
217         return nil, err
218     }
219     defer rows.Close()
220     out := make([]Share, 0)
221     for rows.Next() {
222         var sh Share
223         if err = rows.Scan(&sh.ID, &sh.FileID, &sh.Remaining, &sh.Expires, &sh.Revoked); err !=
224         nil {
225             return nil, err
226         }
227         out = append(out, sh)
228     }
229     return out, rows.Err()
230 }
231 // Consume spends one admitted GET, not one guaranteed completed network download.
232 // If the client disconnects after commit, the ticket is still spent (fail closed).
233 func (s *Store) Consume(ctx context.Context, token string, now time.Time) (File, []byte, error)
234 {

```

```

231     if len(token) != 64 {
232         return File{}, nil, ErrGone
233     }
234     if _, err := hex.DecodeString(token); err != nil {
235         return File{}, nil, ErrGone
236     }
237     tx, err := s.db.BeginTx(ctx, nil)
238     if err != nil {
239         return File{}, nil, err
240     }
241     defer tx.Rollback()
242     var id int64
243     err = tx.QueryRowContext(ctx, `UPDATE shares SET remaining=remaining-1 WHERE token_hash=?
AND revoked=0 AND expires_at>? AND remaining>0 RETURNING file_id`, digest([]byte(token)),
now.Unix()).Scan(&id)
244     if errors.Is(err, sql.ErrNoRows) {
245         return File{}, nil, ErrGone
246     }
247     if err != nil {
248         return File{}, nil, err
249     }
250     var f File
251     var data []byte
252     err = tx.QueryRowContext(ctx, `SELECT f.id,f.name,f.hash,f.size,f.created_at,b.data FROM
files f JOIN blobs b ON b.owner=f.owner AND b.hash=f.hash WHERE f.id=?`, id).Scan(&f.ID,
&f.Name, &f.Hash, &f.Size, &f.Created, &data)
253     if err != nil {
254         return File{}, nil, fmt.Errorf("load admitted file: %w", err)
255     }
256     if err = tx.Commit(); err != nil {
257         return File{}, nil, err
258     }
259     return f, data, nil
260 }

```

internal/httpapi/api.go

```

001 package httpapi
002
003 import (
004     "crypto/subtle"
005     "encoding/json"
006     "errors"
007     "github.com/xr-xiaoran/pocketvault/internal/vault"
008     "io"
009     "mime"
010     "net/http"
011     "strconv"
012     "strings"
013     "time"
014 )
015
016 // Keys map trusted owner labels to operator-provisioned API keys. Never trust an X-User header.
017 func Router(s *vault.Store, keys map[string]string) http.Handler {
018     mux := http.NewServeMux()
019     mux.HandleFunc("GET /health", func(w http.ResponseWriter, r *http.Request) {
020         if s.Ping(r.Context()) != nil {
021             write(w, 503, map[string]string{"error": "database unavailable"})
022             return
023         }
024         write(w, 200, map[string]string{"status": "ok"})
025     })
026     mux.HandleFunc("GET /s/{token}", func(w http.ResponseWriter, r *http.Request) {
027         if r.Method != http.MethodGet {
028             write(w, 405, map[string]string{"error": "only GET consumes a ticket"})
029             return
030         }
031         if r.Header.Get("Range") != "" {
032             write(w, 416, map[string]string{"error": "range downloads are not supported"})
033             return
034         }
035         f, data, err := s.Consume(r.Context(), r.PathValue("token"), time.Now())
036         if err != nil {
037             fail(w, err)
038             return
039         }
040         w.Header().Set("Content-Type", "application/octet-stream")
041         w.Header().Set("Content-Disposition", mime.FormatMediaType("attachment",
042             map[string]string{"filename": f.Name}))
043         w.Header().Set("Content-Length", strconv.Itoa(len(data)))
044         w.Header().Set("Cache-Control", "no-store, private")
045         w.Header().Set("X-Content-Type-Options", "nosniff")
046         w.Header().Set("Referrer-Policy", "no-referrer")
047         _, _ = w.Write(data)
048     })
049     mux.Handle("/api/", http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
050         parts := strings.Fields(r.Header.Get("Authorization"))
051         if len(parts) != 2 || !strings.EqualFold(parts[0], "Bearer") {
052             write(w, 401, map[string]string{"error": "unauthorized"})
053             return
054         }
055         token := parts[1]
056         owner := ""
057         for label, key := range keys {
058             if len(key) >= 16 && subtle.ConstantTimeCompare([]byte(token), []byte(key)) == 1 {
059                 owner = label
060             }
061         }
062         if owner == "" {

```

```

062         write(w, 401, map[string]string{"error": "unauthorized"})
063         return
064     }
065     private := http.NewServeMux()
066     private.HandleFunc("POST /api/files", func(w http.ResponseWriter, r *http.Request) {
067         data, err := io.ReadAll(http.MaxBytesReader(w, r.Body, vault.MaxBytes))
068         if err != nil {
069             write(w, 413, map[string]string{"error": "file exceeds 4 MiB"})
070             return
071         }
072         f, dedup, err := s.Put(r.Context(), owner, r.URL.Query().Get("name"), data,
time.Now())
073         if err != nil {
074             fail(w, err)
075             return
076         }
077         write(w, 201, map[string]any{"file": f, "deduplicated": dedup})
078     })
079     private.HandleFunc("GET /api/files", func(w http.ResponseWriter, r *http.Request) {
080         v, err := s.List(r.Context(), owner)
081         if err != nil {
082             fail(w, err)
083             return
084         }
085         write(w, 200, v)
086     })
087     private.HandleFunc("GET /api/stats", func(w http.ResponseWriter, r *http.Request) {
088         v, err := s.Stats(r.Context(), owner)
089         if err != nil {
090             fail(w, err)
091             return
092         }
093         write(w, 200, v)
094     })
095     private.HandleFunc("DELETE /api/files/{id}", func(w http.ResponseWriter, r
*http.Request) {
096         id, err := strconv.ParseInt(r.PathValue("id"), 10, 64)
097         if err != nil {
098             fail(w, vault.ErrInvalid)
099             return
100         }
101         if err = s.Delete(r.Context(), owner, id); err != nil {
102             fail(w, err)
103             return
104         }
105         w.WriteHeader(204)
106     })
107     private.HandleFunc("POST /api/files/{id}/shares", func(w http.ResponseWriter, r
*http.Request) {
108         id, err := strconv.ParseInt(r.PathValue("id"), 10, 64)
109         if err != nil {
110             fail(w, vault.ErrInvalid)
111             return
112         }
113         var in struct {
114             Downloads int `json:"downloads"`
115             TTL       int `json:"ttl_seconds"`
116         }
117         d := json.NewDecoder(http.MaxBytesReader(w, r.Body, 2048))
118         d.DisallowUnknownFields()
119         if d.Decode(&in) != nil || d.Decode(&struct{}{}) != io.EOF {
120             fail(w, vault.ErrInvalid)
121             return
122         }

```

```

123         sh, err := s.Share(r.Context(), owner, id, in.Downloads, in.TTL, time.Now())
124         if err != nil {
125             fail(w, err)
126             return
127         }
128         write(w, 201, sh)
129     })
130     private.HandleFunc("GET /api/shares", func(w http.ResponseWriter, r *http.Request) {
131         v, err := s.Shares(r.Context(), owner)
132
133         if err != nil {
134             fail(w, err)
135             return
136         }
137         write(w, 200, v)
138     })
139     private.HandleFunc("DELETE /api/shares/{id}", func(w http.ResponseWriter, r
140 *http.Request) {
141         id, err := strconv.ParseInt(r.PathValue("id"), 10, 64)
142         if err != nil {
143             fail(w, vault.ErrInvalid)
144             return
145         }
146         if err = s.Revoke(r.Context(), owner, id); err != nil {
147             fail(w, err)
148             return
149         }
150         w.WriteHeader(204)
151     })
152     private.ServeHTTP(w, r)
153 }
154 func write(w http.ResponseWriter, status int, v any) {
155     w.Header().Set("Content-Type", "application/json; charset=utf-8")
156     w.Header().Set("Cache-Control", "no-store")
157     w.WriteHeader(status)
158     _ = json.NewEncoder(w).Encode(v)
159 }
160 func fail(w http.ResponseWriter, err error) {
161     status := 500
162     message := "internal error"
163     switch {
164     case errors.Is(err, vault.ErrInvalid):
165         status = 400
166         message = err.Error()
167     case errors.Is(err, vault.ErrNotFound):
168         status = 404
169         message = err.Error()
170     case errors.Is(err, vault.ErrGone):
171         status = 410
172         message = err.Error()
173     case errors.Is(err, vault.ErrQuota):
174         status = 409
175         message = err.Error()
176     }
177     write(w, status, map[string]string{"error": message})
178 }

```